Machine Learning with Applications 20 (2025) 100639

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

Software engineering meets legal texts: LLMs for auto detection of contract smells☆

Moriya Dechtiar [a,*], Daniel Martin Katz [b,c,d], Hongming Wang [a]

[a] Harvard University, Cambridge, 02138, MA, USA
[b] Illinois Tech-Chicago Kent College of Law, Chicago, 60616, IL, USA
[c] CodeX - The Stanford Center for Legal Informatics, Stanford, 94305, CA, USA
[d] Center for Legal Technology & Data Science, Bucerius Law School, Hamburg, 20355, Germany

ARTICLE INFO

Keywords:
LLMs
Legal AI
Hyper-parameters fine tuning
BERT
Contract smells
Semi-automated labeling

ABSTRACT

Although there have been many major advances in Artificial Intelligence including its application to a wide variety of tasks, some specialized domains remain difficult to tackle. In this work, we examine parallels between software engineering and legal contract drafting and analysis. Porting well-known code smells principles to various legal contracts, we introduce "contract smells," text patterns that are indicative of potentially significant issues within contractual agreements. We leverage semi-auto labeling with GPT-4, prompting and expert spot checks, to create datasets for suitability testing of auto detection of these contract smells. Using transformer-based models, we explore the impact of legal domain knowledge, hyperparameters fine tuning and specific task information on detection success. We achieve high accuracy with further fine-tuning of BERT as well as LEGAL-BERT, while more consistent results were achieved using task-specific data. We further demonstrate that although multi-class detection can boost coverage of rare smells, single-class detection yields better accuracy. While this is an initial foray into the idea of contract smells, this work underscores the feasibility of applying advanced NLP techniques and LLMs to automate aspects of legal contract review, suggesting a scalable path toward standardized, machine-assisted legal drafting and analysis.

1. Introduction

In software engineering, there is a well-known concept of "code smells" which are easily detected code drafting patterns that provide a good indication of deeper problems within a particular software system. Early detection of such issues significantly improves system maintainability. Furthermore, it prevents major repair and editing costs when otherwise identifiable defects are deployed in widely used software systems. Several works such as (Gupta, Gulanikar, Kumar, & Neti, 2021) demonstrated the successful detection of these code smells within a software system using machine learning and NLP techniques.

Although not completely analogous, there are interesting parallels between software development and contract drafting. For example, in the case of contract drafting, much like in the deployment of software, it is far easier to remedy or debug problems in agreements during the drafting stage than after contract execution. Once the contract is signed every change or potential repapering can impose major costs (Cohen, Schell, Gummaregula, & Pugh, 2021).

Contracts are difficult texts to draft, read, and interpret, even for experienced legal professionals (Hagedoorn & Hesen, 2009). Internal references, external dependencies, constantly changing regulations, and the need to articulate a wide range of contingencies all make this class of documents extremely challenging (Martínez, Mollica, & Gibson, 2022). A lack of precision in contract drafting can lead to disputes, interventions and even lengthy and expensive litigation ((Cheung & Pang, 2013), Maggs (2002)).

Given their tremendous impact on individuals and organizations, AI-driven approaches have sought to ease the burden of contract drafting as well as contract review and analysis through clause extraction, named entity recognition and other related tasks. ((Glaser, Waltl, & Matthes, 2018), Bommarito, Katz, and Detterman (2021), Chalkidis, Androutsopoulos, and Aletras (2021), Sainani, Anish, Joshi, and Ghaisas (2020)) Despite these useful contributions, there are still meaningful opportunities to more systematically detect broader issues in contracts.

Early identification of code smells brought standards to a non regularized industry, improved software quality, streamlined development process and saved money and effort on defect troubleshooting and resolution. Therefore, the concept of "smells" in software code has been

☆ Source code for this work is available on Github, datasets are available on HuggingFace.

* Corresponding author.
E-mail addresses: moriya.dechtiar@alumni.harvard.edu (M. Dechtiar), dkatz3@kentlaw.iit.edu (D.M. Katz), hw@fas.harvard.edu (H. Wang).

https://doi.org/10.1016/j.mlwa.2025.100639
Received 21 January 2025; Received in revised form 25 February 2025; Accepted 12 March 2025
Available online 18 March 2025
2666-8270/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

widely adopted in the software engineering industry to the point of becoming a strong indication of software quality. Code smells flagging is now integrated in software development training, best practices, and is even embedded within code development editors for real-time identification of such issues.

Following (Coupette, Hartung, Beckedorf, Böther, & Katz, 2023), in this paper, we port and apply code smells principles to smells in contractual agreements. While this is an initial effort, we believe the approach we begin to chart herein, can potentially bring real benefits to the legal sector, reducing risk, promoting best practices standards and quality improvement in contract drafting, leading to impactful benefits to individuals, professionals, organizations, and even the justice system itself.

2. Related work

Code smells focuses on code manifestations or patterns that could be indications of a deeper problem within the software system. Code smells were introduced in the late 1990s (Fowler, 1999) and slowly gained popularity among software engineers (Fowler, 2018). Since then, the concept has been further refined into clean code writing guidelines and standards.

Early identification of code smells are extremely beneficial in software development and maintenance. To help automate their identification, machine learning and other allied methods have been developed to improve the smell detection process. Di Nucci, Palomba, Tamburri, Serebrenik, and De Lucia (2018), Fontana, Mäntylä, Zanoni, and Marino (2015), Guggulothu and Moiz (2020), Gupta et al. (2021), Maneerat and Muenchaisri (2011). For example, Gupta et al. (2021) evaluated the effectiveness of natural language processing (NLP) methods and other machine learning (ML) methods for predicting code smells. In this research, ML techniques were used to predict eight different code smells. The findings suggest that different datasets and ML techniques performed better for different types of code smells, indicating that there is no one model fits all approach that could be effective for catching all code smells. The work demonstrated the potential of ML/NLP methods for automating code smell detection to aid developers. Another research paper by Sharma, Efstathiou, Louridas, and Spinellis (2019) explored the feasibility of transfer learning for code smells using deep learning, demonstrating how using RNNs and CNNs for code smell detection is effective and can even be transferred between different code languages. Guggulothu and Moiz (2020), by contrast, outlined another approach for code smell detection using multi-label classification. This approach takes into account the level at which a code smell manifested. Leveraging a method-level dataset, the researchers also redefined code smell detection as a multilabel classification challenge. This approach showed better results increasing the accuracy of detecting code smells. Collectively, these studies (Di Nucci et al., 2018; Fontana et al., 2015; Guggulothu & Moiz, 2020; Gupta et al., 2021; Maneerat & Muenchaisri, 2011) demonstrated the successful use of different ML architectures in order to auto-detect issues in code patterns.

Porting code smells to additional drafting tasks or structured text is very much possible with the help of ML/NLP. Soares et al. (2023) explored test smells which are smells that exist in software testing scripts. This work developed definitions for "test smells" and then created a tool to automatically detect these smells within a dataset of test scripts. A very recent article leveraged ML for automatic detection of smells, this time focusing on defining API documentation smells and using ML for automatic detection of those smells (Khan, Khondaker, Uddin, & Iqbal, 2021). The authors created the documentation smells dataset targeting five types of API documentation smells that they have defined. Multiple automated detection techniques were used and the top performer was BERT which achieved an F1 score of 0.75–0.97. Overall, Khan et al. (2021) demonstrates BERT's language capabilities and its proven potential to understand code smells definitions and further extend on them to understand related issues that occur in different types of texts.

Building upon this and other related work, Coupette et al. (2023) began the task of translating concepts from software engineering to the legal domain. Leveraging concepts from the code smells literature to describe issues with legislative texts, the authors identified the similarities between these seemingly different domains. Law smells were thus defined as easily detected issues in laws and regulations that can indicate deeper problems in the law. The paper successfully identified several such smells and developed a toolkit to demonstrate how to detect some of these smells in US laws. Coupette et al. (2023) is the inspiration for further translating law smells into contract smells. In this work, we define contract smells, and explore the potential of transformer based auto detection of these contract patterns.

Our work also builds upon recent advances in machine learning and NLP within the legal domain. Katz, Dolin, and Bommarito (2021) Namely, although legal language is complex and so in turn are legal documents, Zhong et al. (2020) provides an overview of the possible benefits NLP can bring to the legal system, including the potential of AI to improve efficiency and accuracy of legal work. The academic literature has also seen a material increase in the quantity and quality of technical legal NLP related scholarship (Katz, Hartung, Gerlach, Jana, & Bommarito, 2023).

In Bommarito et al. (2021), the authors develop an open-source library for NLP and machine learning, LexNLP, which is focused upon some of the specific document related challenges for NLP in the legal domain. In Chalkidis, Fergadiotis, Malakasiotis, Aletras, and Androutsopoulos (2020) the authors introduced Legal-BERT, which is a domain-specific BERT model that was pre-trained on legal texts to help support various legal tasks including contract element extraction. In Katz, Bommarito, Gao, and Arredondo (2024), the authors demonstrated that large language models could pass the Bar Exam while recent work has demonstrated material progress on key tasks in fields such as tax law (Nay et al., 2024), patent law (Wang et al., 2024) and regulatory compliance (Berger et al., 2023).

Similar to the broader field of NLP, scholars in Legal NLP have proposed a variety of benchmarks ((Guha et al., 2024), Niklaus et al. (2023), Chalkidis et al. (2022)). There has also been specific work on benchmarking contract review and analysis. Hendrycks et al. (2021) In addition, although not specifically following the "contract smells" approach, a number of recent papers have specifically explored how elements of contracts analysis might be enhanced using machine learning and NLP. ((Narendra, Shetty, & Ratnaparkhi, 2024), Lam, Cheng, and Yeong (2023), Koreeda and Manning (2021), Chalkidis et al. (2021), Hassan and Le (2020)).

3. From code smells to contract issues

3.1. Parallels between software development and legal contract drafting

Although not identical, we can draw some significant parallels between legal contract drafting and software development. These similarities in structure, process and usages, are the basis for selection of potential code smells that can be ported and adapted to become contract smells.

i. Logical structure: Both tasks require sticking to a logical structure. This structure has a significant role in leading to and verifying the expected result. In software the structure implements the control flow, conditions and outcomes. In legal contracts, the structure not only organizes the logical flow of the clauses but also details responsibilities and covers various conditions and contingencies. Both have interdependent components as well as hierarchy. This parallel allows us to adapt code smells related to structure and dependencies into contract smells.

ii. Clarity: Since both rely on textual descriptions and in order for both to achieve exactly the desired results, software code and legal contracts should ideally be very clear and contain no ambiguity.[1]
iii. Maintenance: Software code and legal contracts need to be adjusted while still in use. Software may need updates for new features, security, or efficiency and improvements. Contracts might be updated to reflect changes in law, business arrangements, or other circumstances. This parallel allows us to adapt code smells related to maintainability challenges and regular modifications into contract smells.
iv. Interpretation for execution: In both cases, the text is being interpreted and then executed based on that interpretation. The impact of a possible inaccurate interpretation could be very significant in both case, resulting in major consequences and losses to the involved parties or software users.
v. Development Life-cycle: In both contract and software drafting, there is a similar set of stages: Requirement gathering – > Design & drafting – > Peer review – > Acceptance testing – > Refining – > Approval & Production Deployment.

3.2. Contract smells definitions

Code smell is defined as an indication of a problem, suggesting a weakness in the code design or implementation, that may increase the risk of future software failures or defects (Fowler, 2018). (Coupette et al., 2023) defined a Law smell as a pattern in legal texts that pose threat to the comprehensibility and maintainability of the law. Similarly, contract smell can therefore be defined as an indication in a legal contract document, that suggests a problem with the contract and increases the risk of a contract failure in comprehensibly and maintainability and may require mitigation or further legal interventions.

3.3. Smell types: Translating software quality metrics to legal contract analysis

In order to develop definitions for possible contract smells, we have presented legal experts with original definitions and examples of common code smells, and requested them to select the ones that could be applied to legal contracts. After selecting the code smells candidates, the experts were requested to redefine these smells in the way they could be manifested in legal contracts. Consolidations of these expert definitions concluded in the below subset demonstrating core code smells and their mappings into the realm of legal contracts. It is important to note that these definitions are not exhaustive and could be potentially further fine tuned in future research.

i. Long Method - Overly Long Clauses. In code, methods that carry on too long can become difficult to parse and manage. Similarly lengthy clauses in legal contracts tend to introduce complexity leading to potential misinterpretations or issues enforcing them.
ii. Duplicate Code - Redundant Language. Redundant Language is common when cutting and pasting codes, or when multiple hands drafting the same long contract and could cause duplication go unnoticed. Repetitions cause not only an unnecessary increase in length and complexity, but can also increase the challenge of maintaining the contract when a clause that has multiple inclusions requires updating.
iii. Temporary Variable - Inconsistent Terms. Variables or fields that are used only temporarily make code harder to follow. Terms or definitions in contracts that arbitrarily shift from section to section can cause similar confusion.

[1] We acknowledge there is a degree of debate in the literature regarding whether parties seek "strategic ambiguity". (Bernheim & Whinston, 1998)

Table 1
Selected contract smells.

Category	Granularity level
Overly Long Clauses (Long Method)	Clause
Inconsistent Terms (Temporary Field)	Contract
Redundant Language (Duplicate Code)	Clause, Contract
Excessive Legalese (Comments)	Clause
Superfluous Clauses (Lazy Class)	Clause, Contract
Inappropriate Intimacy	Clause, Contract

iv. Comments - Excessive Legalese. Over-reliance on comments inside code segments to explain complex code can be mapped to using excessive legalese or legal jargon terms that can in fact make the contract harder to understand not only for the general public but also for less experienced law professionals.
v. Lazy Class - Superfluous Clauses. Code that lacks specificity or detail, or clause that add no real legal weight as it is not detailed enough to be enforceable or effective, will only make the contract longer, more complex to understand.
vi. Inappropriate Intimacy - Overly Detailed Disclosures. In code, classes coupled too tightly run risks. Contracts often include sensitive information, overly detailed disclosures could cause a breach of confidentiality and an increased liability.

4. Methodology

Source code for this work is available on Github, datasets are available on HuggingFace.

4.1. Contract smells selection

Contract smells can be identified within a sentence, a clause, or an entire contract, based on their nature and the amount of information required to identify them. Some contract smells can exist at several different levels of text granularity. Considering model limitations, when assessing the suitability of different contract smells for auto detection by machine learning models, we identify the possible levels for each contract smell. Then, we select the smells that can be identified on a clause level, as opposed to a higher degree of granularity, as this better balances both performance and value. See Table 1 for contract smells candidates and their respective levels.

4.2. Datasets preparation

Our training datasets were created from a collection of contracts used in CUAD. Hendrycks and Burns and Chen and Ball (2021) This collection comprises over 500 legal contracts, encompassing 25 different types of contracts. These contracts vary in length, ranging from a few pages to over 100 pages. The annotations produced during CUAD work are not relevant for the purpose of this research, however the store of contracts that are publicly available are very useful as obtaining legal contracts can be challenging due to privacy and confidentiality considerations.

It is important to note that from a quality perspective, the CUAD dataset is a collection of contracts that are not designed to be a representative set. Rather, these are executed agreements that have reached the end of their respective "development life-cycle" and were made available to help support various ML/NLP tasks such model-tuning. From a software engineering perspective, these contracts are akin to a live "production system". Therefore, if anything, the results we report should be seen as the lower bound from a future utility perspective. In other words, we believe the approach we present herein will be even more helpful in real-world contracting workflows, as most contracts encountered outside of controlled datasets are likely to be of lower quality and therefore require more remediation than those included in the CUAD dataset.

In order to train models to detect contract smells that were introduced above, we needed to create a labeled dataset of contract clauses from the CUAD collection of contracts. The dataset columns are: contract-id, clause, LongMethod, DataClumps, TempField, DuplicateCode, and ExcessLegalese. Obtaining manual labeling by law professionals is expensive and extremely time consuming, and so for this work a combination of automatic labeling and human labeling were both utilized. Automatic labeling process:

i. The model selected for the task of automatic labeling was the SOTA model at that time that also offered API access, GPT4-turbo by OpenAI (Achiam et al., 2023).
ii. As prompt has a powerful impact on the responses we obtain from the selected model, we decided to construct several slightly different prompts and evaluate their contribution to the models success. Building on the proven success reviewed in our related work section, we see ML models are able to classify code smells efficiently and with high accuracy. Therefore 3 prompts were constructed: Our first prompt of a technical code perspective, used the pure code smells definitions as they are used in the context of software development. Second prompt of a more textual perspective, used the definitions we created earlier after mapping these code smell definitions into the realm of legal contracts. A third prompt was constructed with a combined approach to include both code smell definition as well as its more textual mapping to produce an extensive definition that also demonstrates the mapping logic.
iii. A total of 3 datasets were produced, using the above mentioned prompt engineered in an extractive zero shot approach as no examples are provided to the model. They datasets are respectively referred to as Textual-Oriented dataset, Coding-Oriented dataset and Extended Auto-Labeled dataset.
iv. Label Validation. We conducted an expert spot checking of a sample of our labeled clauses, to ensure the automatically generated labels aligned with our intended definitions of contract smells. This process was not done to test GPT-4's accuracy compared with human accuracy, rather, it was a sanity check to validate that the labels were mapped reasonably based on the prompt definitions approach used to create them. A team of 3 legal experts, each with at least 3 years of legal contracts experience, was asked to review and validate 7% of our labeled clauses. Upon disagreements, further discussion took place and if disagreement remained, the majority vote was the one applied. Less than 10% of sampled clauses had a disagreement upon initial individual expert review. Once legal professionals verified that the mappings made sense based on our contract-smell definitions, we proceeded with standard dataset cleaning (e.g., removing duplicates, empty lines, discarding malformed clauses).

While the GPT-4 has demonstrated real promise on certain law related tasks (Katz et al., 2024), it is important to make note of GPT-4 limitations for analyzing legal texts. In terms of domain specific terminology, GPT-4 may struggle with legal terminology and while its training data does contain some legal texts, it is unclear if this data is sufficient to perform an analysis that is comparable to human expert performance. GPT-4 also could be missing legal context of anything beyond the specific contract it is presented with. GPT-4 also truncates based on the context window which could impact its ability to analyze longer texts. In order to minimize the impact of these limitations, we have selected a subset of contract smells that require the least amount of legal context. Also, as mentioned above, we have spot checked clauses labeled by GPT-4 and expert-validated its ability to understand and flag each of our selected contract smells. That said, testing GPT-4 abilities is outside of the scope for this research which is focused on the development of contract smells definitions and proving the feasibility of detecting them using deep learning.

4.3. Model selection

In order to achieve the best possible results for detecting contract smells, several experiments were conducted on 3 different LLMs, all based on BERT which has proven to perform well in both code smells detection task and in legal domain tasks:

i. Baseline BERT. Our baseline model is representing general-purpose models. BERT is run on each dataset and produces a prediction on each of the contract smells in question. This model has no prior knowledge of the domain, nor does it have the full context of the contract. This model is trained and evaluated on all 3 datasets. We used the "bert-base-uncased" checkpoint from the Hugging Face Transformers library (Devlin, Chang, Lee, & Toutanova, 2019).
ii. LEGAL-BERT. This model is selected to represent domain-specific models and will allow evaluation of prior general domain knowledge and how it is contributing to anomalies detection. The LEGAL-BERT model was trained on a variety of legal text and is able to capture the meaning of legal content. This experiment is to determine the significance of the domain knowledge on the ability to detect contract smells. This model is trained and evaluated on all 3 datasets. The hugging face model used is: "nlpaueb/legal-bert-base-uncased" (Chalkidis et al., 2020). This gives the model broader legal text exposure.
iii. LEGAL-BERT CUAD Fine Tuned. This model was selected to represent task-specific fine-tuning. This experiment is evaluating the ability to detect contract smells when the model has both pretaining of legal domain as well as specific training on the contracts from which the labeled data is taken from. This is expected to show the best results for detecting contract smells. This model is trained and evaluated on all 3 datasets. We used the open-source variant from the CUAD work (Hendrycks & Burns & Chen & Ball, 2021), which was fine-tuned on legal contract QA tasks. The hugging face model was created by fine tuning "nlpaueb/legal-bert-base-uncased" on the CUAD Q&A dataset to allow our model better understanding of the specific contracts our smells were taken from.

4.4. Model training

The above selected models were trained on the datasets created. Each labeled dataset was split 80/10/10 for training, validation, and testing sets. The first set of training was naive, i.e. training with no special consideration to the classes distributions and using only a single set of standard parameters and a single contract detection problem framing - training on each contract smell separately. After the naive training concluded, we ran a set of additional training experiments incorporating the following strategies:

i. Systematically tuned hyper parameters: batch sizes (8, 16, 32), learning rate that were selected from range: [1e-5, 5e-4], and epochs from range [2,6].
ii. Two sets of detection problem framing, single smell classification, and multi smell classification to try and leverage knowledge captured by smells relations or dependencies.
iii. We paid particular attention to rare classes by analyzing confusion matrices and adjusting class weights in our network forward pass.

4.5. Evaluation metrics

To evaluate the performances of the above selected NLP models for detecting the contract anomalies that were defined, we leveraged a set of standard metrics widely used in both software quality assessment and natural language processing tasks. These metrics allow us to quantify the models' performance across different types of contract issues and compare across models, datasets, and contract smells types.

i. Accuracy: Is the primary metric to measure the overall correctness of our models. This metric calculates how many correct predictions we have made compared to the total number of samples assessed. This provides a good general indication but could be misleading for imbalanced datasets, which is one of challenges for anomalies detection task.
ii. Precision and Recall: Calculating precision and recall for each anomaly type helps complete the picture and compensates for the challenge mentioned above for accuracy as a sole metric. These metrics are particularly important in legal document analysis, where false positives (flagging non-issues) and false negatives (missing real issues) can have serious consequences and damages.
iii. F1 Score: The harmonic mean of precision and recall. This metric is most useful when comparing performances on imbalanced datasets, and of rare occurring samples.
iv. AUC Score: Calculating Area Under the ROC Curve helps evaluate performance of models in identifying the positive class (i.e. a positive for contract issue). As the legal consequence of a contract issue can be serious, we would prefer higher AUC to achieve more true negatives even at the expense of having more false positives.

These are calculated across all models, datasets and contract anomalies, in order to capture not only the performance of the model, but also the contribution of the auto labeling method and the potential each type of anomaly has to be effectively detected by machine learning. Another important note is since the impact of a contract failure, i.e., a contract smell that was missed, is significant and can translate into damages, we have focused on the accuracy but also on the recall in this paper.

5. Experimental results and analysis

5.1. Auto labeling

The datasets that were created on specific prompt definitions are showing some similarities in class compositions for most of the contract anomalies in question. LongMethod and ExcessLegalese are both classes that could be over represented in both datasets which impacts models tendency to classify clauses for these classes. In contrast to DuplicateCode, TempField that may be underrepresented. InappropIntimacy and LazyClass are both even more rare in our Extended dataset. These differences can be explained by the impact of rare events i.e. because contract smells are generally rare, and most contracts examined are of high quality, there is more impact to the prompt provided as there are simply less examples to learn from. Since we have this imbalance in the datasets, it is more reasonable to train separately on each contract smell than it would be to train for a multi label problem that could result in skewed results. Fig. 1 displays the dataset class composition and distribution of contract smells. Fig. 2 visualizes the co-occurrence of classes matrix.

5.2. Naive training

5.2.1. BERT naive training

Surprisingly, BERT provided a strong baseline performance. We observe that it often outperforms or matches the performance of the legal models we tested. The strong performance of BERT Baseline suggests that for the task of detecting contract smells, general language understanding capabilities may be more important than domain-specific knowledge which we have in the LEGAL-BERT or CUAD-LegalBERT pretraining.

Tables 2 and 3 list the accuracy and recall scores achieved by BERT for detecting contract smells defined.

Table 2
BERT baseline accuracy analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	89.12%, 91.39%, 94.00%
DataClumps	96.30%, 96.12%, 98.68%
TempField	99.42%, 98.76%, 97.50%
DuplicateCode	98.64%, 99.43%, 98.09%
ExcessLegalese	77.99%, 88.54%, 87.35%

Table 3
BERT baseline recall analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	83.19%, 90.37%, 92.47%
DataClumps	78.46%, 86.29%, 67.64%
TempField	86.23%, 88.93%, 75.55%
DuplicateCode	50.00%, 71.13%, 66.79%
ExcessLegalese	50.00%, 78.73%, 83.76%

Table 4
LEGAL-BERT accuracy analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	83.71%, 73.23%, 93.84%
DataClumps	96.98%, 90.40%, 98.65%
TempField	98.64%, 96.40%, 97.90%
DuplicateCode	99.06%, 99.04%, 98.39%
ExcessLegalese	80.64%, 78.63%, 87.18%

Table 5
LEGAL-BERT recall analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	75.49%, 66.54%, 92.05%
DataClumps	79.61%, 57.06%, 63.49%
TempField	50.00%, 50.00%, 69.83%
DuplicateCode	60.86%, 50.00%, 83.36%
ExcessLegalese	54.03%, 50.00%, 83.75%

5.2.2. Legal-BERT naive training

For this model we can actually see that accuracy scores for the textual dataset are better than the ones obtained on the coding oriented dataset with 83.71% for Long Method compared to 73.23% and 96.98% for Data Clumps compared to 90.4%. We can also see that precision and recall are better on the textual dataset as well with 75.49% for Long Method compared to 65.54% and 79.61% for Data Clumps compared to 57.06%. However, these results are improved significantly when training on the Extended dataset built using the extended definitions. This suggests the Legal context this model has, enabled it to detect contract smells better on a textual definition and improved balance overall. Tables 4 and 5 list the accuracy and recall scores achieved by BERT for detecting contract smells defined.

5.2.3. CUAD legal-BERT naive training

We can see this model also performs well in terms of accuracy across all categories especially on the coding-oriented dataset. In previous models we have seen that while accuracy was high, F1 scores were not as good, with accuracy above 90% but F1 scores that ranges between 70% and 85%. However here we can see improvements on the F1 scores compared to the original LEGAL-BERT, and similar in performance to the baseline BERT model. Tables 6 and 7 list the accuracy and recall scores achieved by CUAD-trained LEGAL-BERT for detecting contract smells defined.

5.2.4. Comparative analysis - naive training

Previous sections displayed the results of training the naive models on each of our contract smells, in a single class binary type of classification. This was using the standard set of configuration with no parameters fine tuning. We can compare the results so far and observe that the specialized legal models (LEGAL-BERT and CUAD-trained

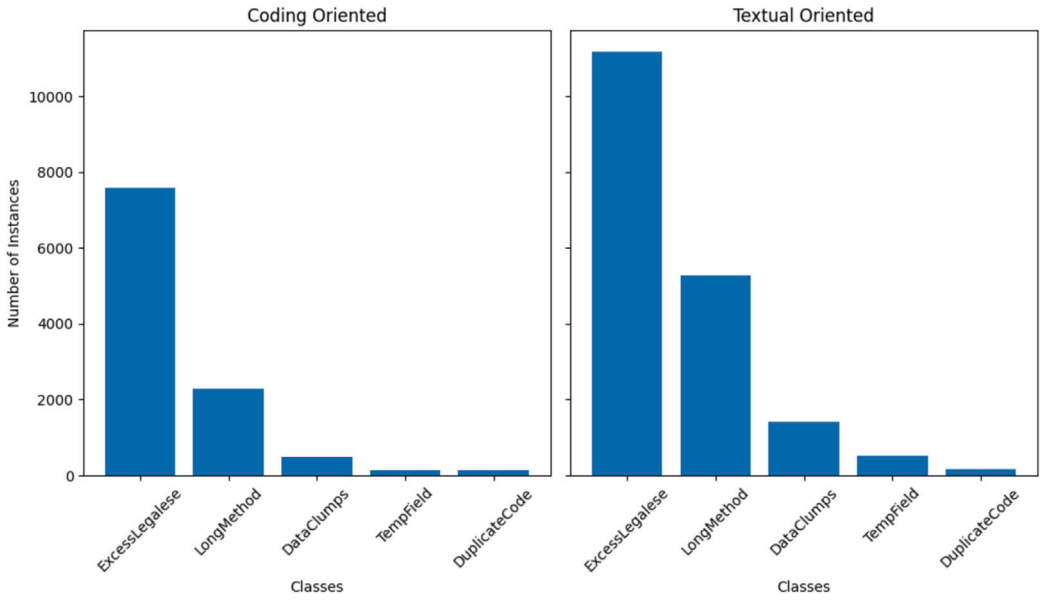


Fig. 1. Classes Distribution Across Datasets.

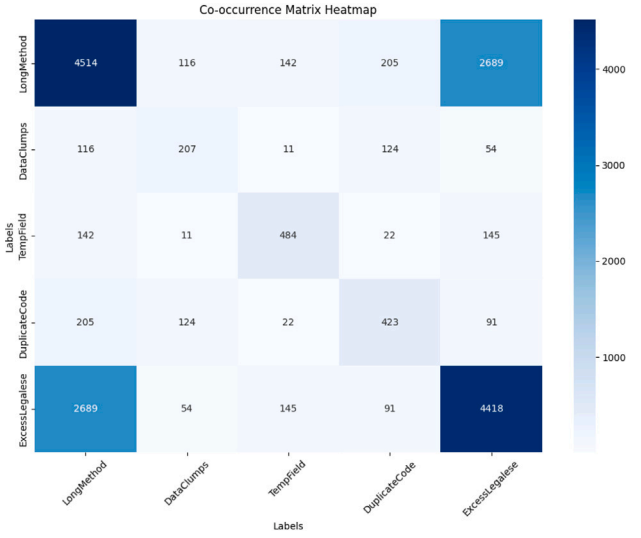


Fig. 2. Classes Co-occurrence Matrix.

Table 6
CUAD trained LEGAL-BERT accuracy analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	84.55%, 89.31%, 93.34%
DataClumps	96.51%, 95.20%, 98.62%
TempField	98.69%, 97.07%, 97.08%
DuplicateCode	98.69%, 99.33%, 98.39%
ExcessLegalese	77.73%, 88.01%, 87.02%

Table 7
CUAD trained LEGAL-BERT recall analysis across contract smells.

Category	Text, Coding, Extended
LongMethod	79.25%, 88.89%, 92.83%
DataClumps	80.20%, 85.64%, 61.84%
TempField	50.00%, 73.99%, 73.69%
DuplicateCode	59.18%, 70.95%, 78.66%
ExcessLegalese	50.00%, 80.01%, 84.21%

Table 8
Average performance metrics by model - naive.

Evaluation metric	BERT	LEGAL-BERT	CUAD
Accuracy text	92.294	91.806	91.234
F1 text	69.576	66.070	63.656
Accuracy code	94.848	87.540	93.784
F1 code	86.068	53.960	82.516
Accuracy extended	95.130	95.196	94.892
F1 extended	79.750	81.000	80.226

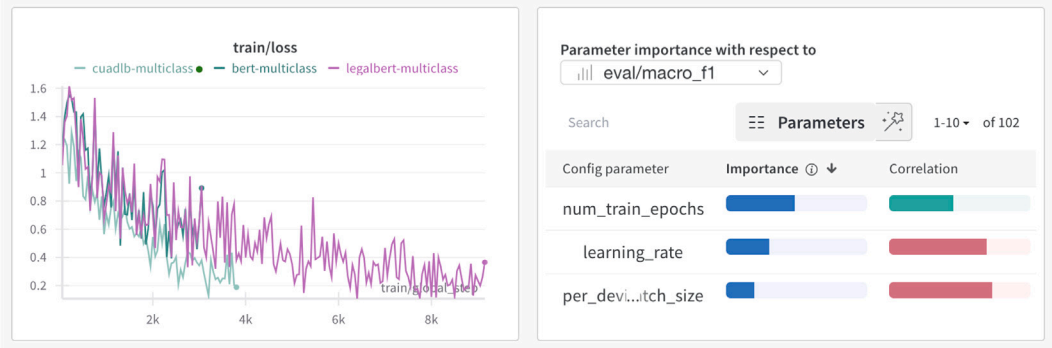
Table 9
Accuracy analysis across models based on smell type.

Contract smell	BERT	LEGAL-BERT	CUAD
LongMethod	94.00%	93.84%	93.34%
DataClumps	98.68%	98.65%	98.62%
TempField	97.50%	97.90%	97.08%
DuplicateCode	98.09%	98.39%	98.39%
ExcessLegalese	87.35%	87.18%	87.02%

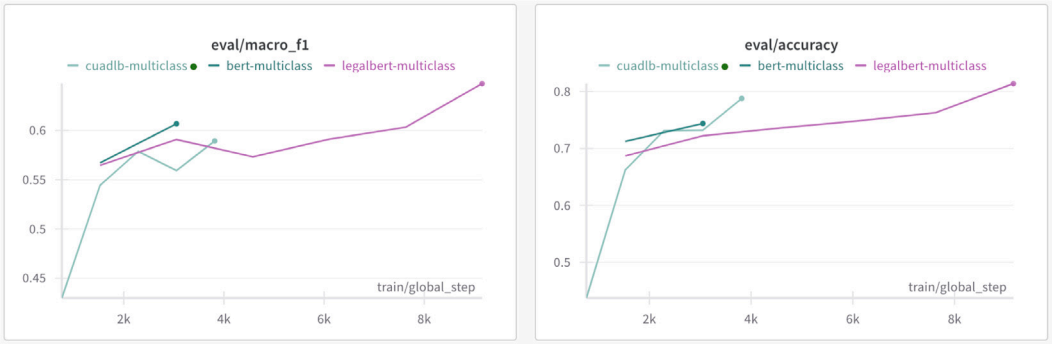
LEGAL-BERT) did not provide a clear advantage in this specific task, however, the CUAD-trained LEGAL-BERT showed more consistency and clearly outperformed LEGAL-BERT. This is suggesting that fine-tuning on a relevant corpus can lead to more robust models that can achieve consistent results across all contract anomalies types. Another important note is the suitability of the model for detecting a specific contract smell. The recall results demonstrate that some contract smells are identified with more success by specific models, others are benefiting more from the legal and specific contract additional context. This is a testament to the differences between contract smells. Some are more complex on the linguistic aspect and require a deeper general language understanding, while others are clearly more legal focused and are showing better results with the CUAD trained LegalBERT. Tables 8 and 9 list the evaluation scores achieved by different models including the F1 scores demonstrating different consistency achieved also based on the different types of contract smells.

5.3. Advanced training

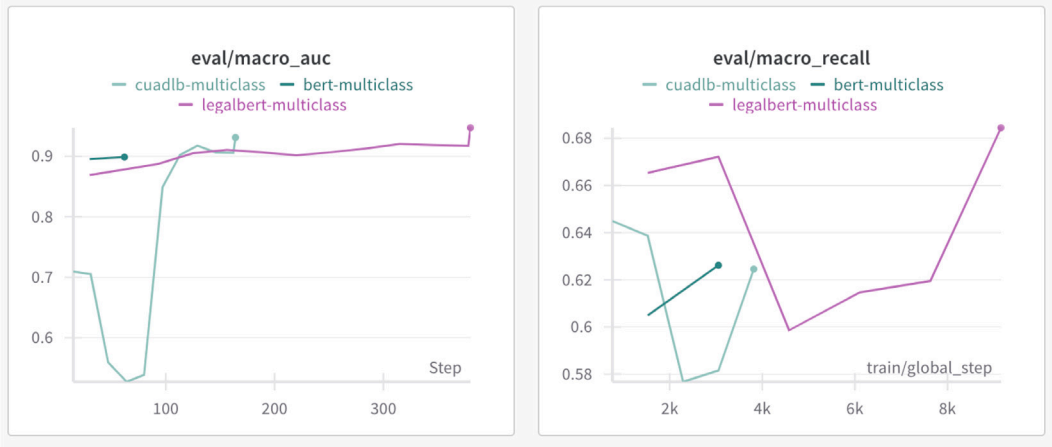
In this part of our training, we use hyper parameters fine tuning, classes adjusted weights and different framing to our detection problem in order to improve on our naive training results.



(a) Multi-class Training Loss & Parameters Impact



(b) Multi-class Performance Metrics



(c) Multi-class AUC and Recall Performance

Fig. 3. Overview of training loss, parameters impact and performance of Multi-class models for smell detection.

5.3.1. Multi-class detection & hyper parameters fine tuning

As contract clauses can contain more than one issue at the same time, our classes are not mutually exclusive, we wanted to examine the potential value of training our models to detect all classes simultaneously. If some issues tend to reoccur, this could even improve performance. In order to provide the models the maximum information possible, this training was run on the extended dataset. The extended dataset was also chosen for this additional examination as it was on this dataset that the highest average F1 score was achieved across our 3 models. This is especially significant metric in context of class imbalances. Fig. 3(a) highlights how well the training loss converges and is consistently decreasing for BERT and LegalBert. CUAD trained LegalBERT is showing the fastest decrease in loss, however it then spikes and loses that advantage compared to the other models. This could be caused by outliers or learning rate schedule that is impacting the results. This figure also includes a visualization of the parameters importance based on our hyperparameters fine tuning runs. Figs. 3(b) and 3(c) highlight the performance metrics calculated. Running hyper parameters fine tuning displayed improvements. We can see that learning rates higher than $2e-4$ are producing close to random performance with almost no actual learning achievement. Smaller rates are converging and producing both better F1 scores as well as high accuracy as shown in Table 10 below. Increasing number of training epochs generally helps, however it depends on the learning rate configured. With higher learning rates, the optimal convergence can be skipped over. In terms of the training batch size, best results are achieved with medium values of 8–16.

It appears that even with HPFT, the multi-class detection is not providing value to our models and a contract smell specific approach

Table 10
Top hyperparameter tuning results for multi-class detection.

Name	Batch	Learn rate	Epochs	F1
cuadlb-multi-1	16	9.60e−05	5	0.589
cuadlb-multi-2	16	2.07e−04	7	0.097
cuadlb-multi-3	8	4.29e−04	3	0.097
cuadlb-multi-4	32	5.77e−05	4	0.559
bert-multi-1	16	2.42e−05	4	0.589
bert-multi-2	32	3.67e−05	4	0.575
bert-multi-3	8	2.39e−05	2	0.606
bert-multi-4	16	6.43e−05	7	0.575
legalbert-multi-1	8	2.88e−05	6	0.647
legalbert-multi-2	32	4.10e−04	2	0.0975
legalbert-multi-3	8	3.41e−05	5	0.573
legalbert-multi-4	8	2.37e−04	3	0.097

Table 11
Top hyperparameter tuning results for single-class detection.

Name	Batch	Learn Rate	Epochs	F1
cuad-legalease	16	2.39e−05	4	0.805
cuad-dupcode	8	5.55e−05	2	0.126
cuad-tempfield	16	2.00e−05	6	0.453
cuad-dataclumps	8	3.62e−05	2	0.090
cuad-longmethod	16	3.75e−05	4	0.913
bert-legalease	16	1.08e−05	7	0.797
bert-dupcode	16	4.58e−05	6	0.551
bert-tempfield	16	1.34e−05	6	0.528
bert-dataclumps	16	1.51e−05	4	0.170
bert-longmethod	16	1.70e−05	5	0.907
lglbert-legalease	32	4.59e−05	2	0.793
lglbert-dupcode	8	2.41e−05	5	0.467
lglbert-tempfield	32	2.89e−05	2	0.500
lglbert-dataclumps	32	2.09e−05	6	0.170
lglbert-longmethod	16	2.87e−05	2	0.906

is preferable. Another observation here is the dominance by LegalBERT showing consistent performance in the multi-class approach.

5.3.2. Single smell detection - hyper parameters fine tuning

To further improve our baseline results for detecting contract smells, we performed experiments to examine how changing major parameters can improve our performances for the different models and smell types. Values were randomly selected for values of learning rate, Fig. 4(a) shows the training loss convergence on the 3 models for each different contract smell type. Fig. 4(b) displays how our hyperparameters changes correlate with the achieved F1 scores. We also visualize the importance of parameters observed during the hyperparameters fine tuning runs.

It appears that fine tuning our hyperparameters, is allowing domain knowledge models to display dominance and results in improved handling of the more difficult to learn classes. Table 11 details the hyper parameters tuning impact of the top runs executed. We can clearly see that learning rate ranges $2e-5$ to $4e-5$ achieve balanced performance with F1 scores. In the higher learning rates, while convergence happen faster, the F1 results on the rarer classes are impacted. Changing our training epochs shows mostly benefits on rarer classes but in most cases 2–4 epochs are sufficient in achieving high accuracy and F1 scores. Lastly, when changing the batch size, we can see that for more balanced classes the value 16 is dominating our top performing runs.

5.3.3. Comparative analysis - multiclass, HPFT

After including class imbalance adjustments of initialized weights, a set of training runs was performed to fine tune hyperparameters in two types of problem definition: multiclass, and single class detection. Following this set of experiments we can identify additional observations. The best performing model runs achieved similar accuracy results for the most part, however the multiclass detection accuracy is showing a significant drop as detailed in Table 12. This suggests that the additional knowledge provided by cross smell type relationships, is not

Table 12
Accuracy metrics for contract smell detection (single & multiclass).

Smell type	BERT	LEGAL-BERT	CUAD
LongMethod	0.9489	0.9587	0.9587
Data Clumps	0.9902	0.9908	0.9895
Temp Field	0.9653	0.9666	0.9705
Duplicate Code	0.9862	0.9836	0.9823
Excess Legalese	0.8814	0.8355	0.8794
Multiclass	0.7438	0.8139	0.7877
Overall Average	0.9544	0.9471	0.9561

Table 13
AUC metrics for contract smell detection (single & multiclass).

Smell type	BERT	LEGAL-BERT	CUAD
LongMethod	0.9825	0.96299	0.98152
Data Clumps	0.7941	0.85157	0.72804
Temp Field	0.87551	0.90545	0.89647
Duplicate Code	0.85884	0.8219	0.65695
Excess Legalese	0.9344	0.93332	0.9345
Multiclass	0.8985	0.94717	0.93091
Overall Average	0.8906	0.9037	0.8547

Table 14
F1 scores for single smell detection.

Smell type	BERT	LEGAL-BERT	CUAD
LongMethod	0.8934	0.9069	0.9101
Data Clumps	0.2857	0.3000	0.2727
Temp Field	0.3457	0.3377	0.4000
Duplicate Code	0.6441	0.5763	0.3077
Excess Legalese	0.7622	0.7125	0.7767

Table 15
F1 scores for multiclass smell detection.

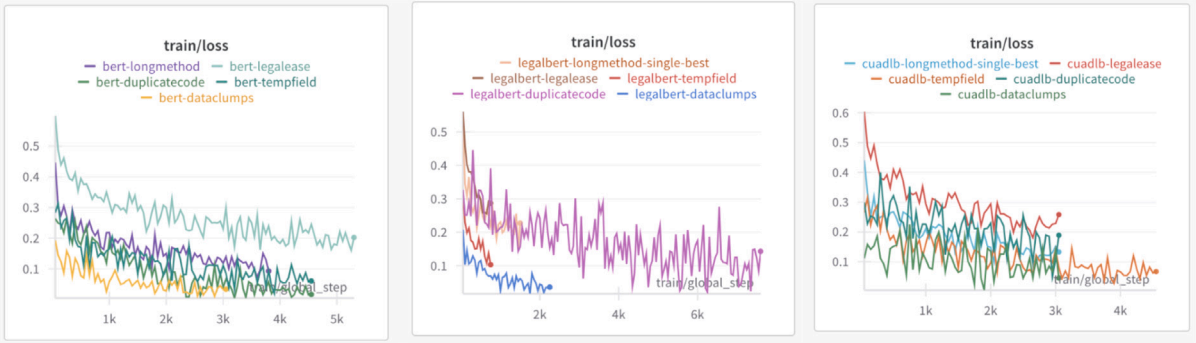
Smell type	BERT-MC	LB-MC	CUAD-MC
LongMethod	0.9068	0.8987	0.8972
Data Clumps	0.3250	0.3846	0.2667
Temp Field	0.4493	0.5347	0.4632
Duplicate Code	0.5532	0.6479	0.5714
Excess Legalese	0.7996	0.7720	0.7482

greater than the specialized training value. Looking at the AUC values detailed in Table 13, LegalBERT multiclass is topping performance with 0.95. Since some of our classes are more rare, it is important to look at the F1 scores as well. Here we can see how our multiclass models have improved the individual smell F1 score in almost all cases. See Tables 14 and 15 for the details on how much F1 scores have improved. Even when inspecting our rare classes in which our models struggled the most, we can see multiclass models F1 scores improve from 0.27–0.3 for Data Clumps to 0.26–0.38, and from 0.33–0.4 for Temp Field to o.45 to 0.53.

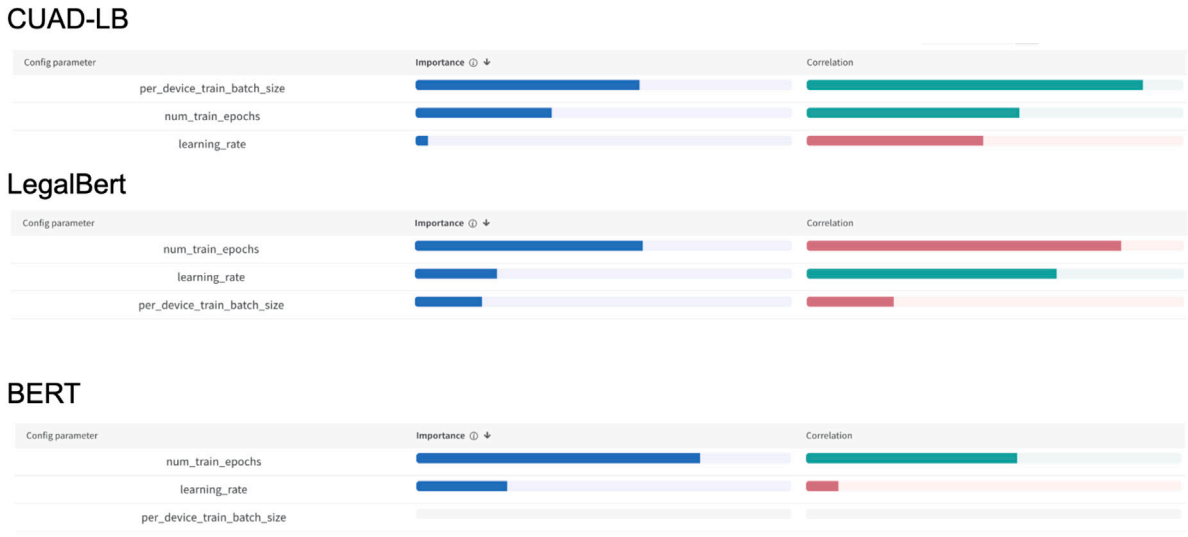
6. Conclusions

In this work, we propose "contract smells", importing software engineering principles to legal contract anomaly detection. We believe this approach offers immense potential for transforming the task of contract review and drafting. Namely, our results highlight the potential for automated contract review to reduce drafting errors, mitigate risk, and accelerate the standardizing of machine-assisted legal error detection. Our achieved results on this current collection of contracts, are even more impactful for real life application, when they can be applied to contracts much earlier in their development life-cycle, when issues are much more prevalent and drastically improve delivery time, effort and contract quality.

In addition, our usage of semi-automated labeled data also suggests a way to improve synthetic data generation and labeling with very



(a) Training Loss for BERT, LegalBERT and CUAD runs



(b) Single Class Detection Model Performance HPFT Impact

Fig. 4. Overview of training loss and parameters impact for single smell detection.

little human input, leveraging the proven capabilities of NLP models in software code understanding.

Our findings demonstrate the potential of interdisciplinary approaches in enhancing legal document quality as well as offering key insights into the contribution of general domain knowledge compared to task specific knowledge in identifying contract smells, and the impact of using different detection approaches. Our multi-class detection experiments also reveal a trade-off between robustness of rare smells handling, and overall accuracy.

Future work should focus on refining and extending contract smells including associated detection methods for subjective anomalies. In addition, both agentic and ensemble models that combine the strengths of general and specialized NLP systems should be explored. Most importantly, we believe that variants of the approach we discuss herein can be applied more broadly to other legal document types and work streams.

CRediT authorship contribution statement

Moriya Dechtiar: Conceptualization, Methodology, Software, Data analysis, Visualization, Writing – original draft. **Daniel Martin Katz:** Supervision, Methodology, Writing – review & editing. **Hongming Wang:** Supervision, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., et al. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.

Berger, A., Hillebrand, L., Leonhard, D., Deußer, T., De Oliveira, T. B. F., Dilmaghani, T., et al. (2023). Towards automated regulatory compliance verification in financial auditing with large language models. In *2023 IEEE international conference on big data (bigData)* (pp. 4626–4635). IEEE.

Bernheim, B. D., & Whinston, M. D. (1998). Incomplete contracts and strategic ambiguity. *American Economic Review*, 902–932.

Bommarito, M. J., Katz, D. M., & Detterman, E. M. (2021). Lexnlp: Natural language processing and information extraction for legal and regulatory texts. *Research Handbook on Big Data Law*, 216–227.

Chalkidis, I., Androutsopoulos, I., & Aletras, N. (2021). Neural contract element extraction. In *Proceedings of the eighteenth international conference on artificial intelligence and law* (pp. 159–168). Association for Computing Machinery, http://dx.doi.org/10.1145/3462757.3466063.

Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). LEGAL-BERT: The muppets straight out of law school. In *Findings of the association for computational linguistics: EMNLP 2020* (pp. 2898–2904).

Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D., et al. (2022). Lexglue: A benchmark dataset for legal language understanding in english. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 4310–4330).

Cheung, S. O., & Pang, K. H. Y. (2013). Anatomy of construction disputes. *Journal of Construction Engineering and Management*, *139*(1), 15–23.

Cohen, M. S., Schell, C. S., Gummaregula, S., & Pugh, J. (2021). Enabling a holistic solution to London interbank offer rate repapering. *Journal of Financial Compliance*, *5*(1), 30–44.

Coupette, C., Hartung, D., Beckedorf, J., Böther, M., & Katz, D. M. (2023). Law smells. *Artificial Intelligence and Law*, [ISSN: 1572-8382] *31*(2), 335–368. http://dx.doi.org/10.1007/s10506-022-09315-w.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies* (pp. 4171–4186). Minneapolis, Minnesota: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/N19-1423, URL https://aclanthology.org/N19-1423.

Di Nucci, D., Palomba, F., Tamburri, D. A., Serebrenik, A., & De Lucia, A. (2018). Detecting code smells using machine learning techniques: Are we there yet? In *2018 ieee 25th international conference on software analysis, evolution and reengineering (saner)* (pp. 612–621). IEEE.

Fontana, F. A., Mäntylä, M., Zanoni, M., & Marino, A. (2015). Comparing and experimenting machine learning techniques for code smell detection. *Empirical Software Engineering*, *21*, 1143–1191, URL https://api.semanticscholar.org/CorpusID:16222152.

Fowler, M. (1999). *Refactoring: improving the design of existing code*. USA: Addison-Wesley Longman Publishing Co. Inc., ISBN: 0201485672.

Fowler, M. (2018). *Refactoring: Improving the design of existing code*. Addison-Wesley Professional.

Glaser, I., Waltl, B., & Matthes, F. (2018). Named entity recognition, extraction, and linking in german legal contracts. In *IRIS: internationales rechtsinformatik symposium* (pp. 325–334).

Guggulothu, T., & Moiz, S. A. (2020). Code smell detection using multi-label classification approach. *Software Quality Journal*, *28*(3), 1063–1086.

Guha, N., Nyarko, J., Ho, D., Ré, C., Chilton, A., Chohlas-Wood, A., et al. (2024). Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in Neural Information Processing Systems*, *36*.

Gupta, H., Gulanikar, A. A., Kumar, L., & Neti, L. B. M. (2021). Empirical analysis on effectiveness of NLP methods for predicting code smell. In *International conference on computational science and its applications* (pp. 43–53).

Hagedoorn, J., & Hesen, G. (2009). Contractual complexity and the cognitive load of R&D alliance contracts. *Journal of Empirical Legal Studies*, *6*(4), 818–847.

Hassan, F. u., & Le, T. (2020). Automated requirements identification from construction contract documents using natural language processing. *Journal of Legal Affairs and Dispute Resolution in Engineering and Construction*, *12*(2), Article 04520009.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., et al. (2021). Measuring massive multitask language understanding. In *International conference on learning representations*.

Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An expert-annotated NLP dataset for legal contract review. In *Thirty-fifth conference on neural information processing systems datasets and benchmarks track (round 1)*.

Katz, D. M., Bommarito, M. J., Gao, S., & Arredondo, P. (2024). Gpt-4 passes the bar exam. *Philosophical Transactions of the Royal Society, Series A*, *382*(2270), Article 20230254.

Katz, D. M., Dolin, R., & Bommarito, M. J. (2021). *Legal informatics*. Cambridge University Press.

Katz, D. M., Hartung, D., Gerlach, L., Jana, A., & Bommarito, M. J., II (2023). Natural language processing in the legal domain. arXiv preprint arXiv:2302.12039.

Khan, J. Y., Khondaker, M. T. I., Uddin, G., & Iqbal, A. (2021). Automatic detection of five api documentation smells: Practitioners' perspectives. In *2021 IEEE international conference on software analysis, evolution and reengineering* (pp. 318–329). IEEE.

Koreeda, Y., & Manning, C. D. (2021). Contractnli: A dataset for document-level natural language inference for contracts. In *Findings of the association for computational linguistics: EMNLP 2021* (pp. 1907–1919).

Lam, K.-Y., Cheng, V. C., & Yeong, Z. K. (2023). Applying large language models for enhancing contract drafting.. In *LegalAIIA@ ICAIL* (pp. 70–80).

Maggs, G. E. (2002). Patterns of drafting errors in the uniform commercial code and how courts should respond to them. *U. Ill. L. Rev.*, 81.

Maneerat, N., & Muenchaisri, P. (2011). Bad-smell prediction from software design model using machine learning techniques. *2011 Eighth International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 331–336.

Martínez, E., Mollica, F., & Gibson, E. (2022). Poor writing, not specialized concepts, drives processing difficulty in legal language. *Cognition*, *224*, Article 105070.

Narendra, S., Shetty, K., & Ratnaparkhi, A. (2024). Enhancing contract negotiations with LLM-based legal document comparison. In *Proceedings of the natural legal language processing workshop 2024* (pp. 143–153).

Nay, J. J., Karamardian, D., Lawsky, S. B., Tao, W., Bhat, M., Jain, R., et al. (2024). Large language models as tax attorneys: a case study in legal capabilities emergence. *Philosophical Transactions of the Royal Society, Series A*, *382*(2270), Article 20230159.

Niklaus, J., Matoshi, V., Rani, P., Galassi, A., Stürmer, M., & Chalkidis, I. (2023). LEXTREME: A multi-lingual and multi-task benchmark for the legal domain. In *Findings of the association for computational linguistics: EMNLP 2023* (pp. 3016–3054).

Sainani, A., Anish, P. R., Joshi, V., & Ghaisas, S. (2020). Extracting and classifying requirements from software engineering contracts. In *2020 IEEE 28th international requirements engineering conference* (pp. 147–157). IEEE.

Sharma, T., Efstathiou, V., Louridas, P., & Spinellis, D. (2019). On the feasibility of transfer-learning code smells using deep learning. arXiv:1904.03031.

Soares, E., Aranda, M., Oliveira, N., Ribeiro, M., Gheyi, R., Souza, E., et al. (2023). Manual tests do smell! cataloging and identifying natural language test smells. In *2023 ACM/IEEE international symposium on empirical software engineering and measurement* (pp. 1–11). IEEE.

Wang, Q., Ni, S., Liu, H., Lu, S., Chen, G., Feng, X., et al. (2024). AutoPatent: A multi-agent framework for automatic patent generation. arXiv preprint arXiv:2412.09796.

Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z., & Sun, M. (2020). How does NLP benefit legal system: A summary of legal artificial intelligence. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 5218–5230).